PYTHON FOUNDATIONS · TOPIC 1

20 MIN

Introducing Python

Idea → Code → Interpreter → Output

Simple syntax

Interpreted

Versatile

Huge libraries

Open source

~3 min

Python Reads Almost Like English

One language, from a first script to a whole data pipeline.

In simple words

Python is a general-purpose programming language created by Guido van Rossum and first released in 1991. It was designed so that code is easy to read, easy to write and quick to try out.

Five ideas, in order

1. **Simple syntax**
2. **Interpreted**
3. **Versatile**
4. **Huge libraries**
5. **Open source**

Fewer lines to say the same thing: that is the thread running through every idea above.

~4 min

What Makes Python Popular

Three design choices that explain most of its success.

Readable syntax

Indentation marks the blocks and there are no braces or semicolons, so code looks like the idea behind it.

Interpreted

Code runs line by line without a separate compile step, so you can test an idea in seconds.

Huge ecosystem

Thousands of free libraries cover maths, web, data and AI, so most problems are already half solved.

Python is free and open source, so the language and its tools cost nothing to learn or use.

~4 min

Your First Program

Three lines: ask, store, answer.

```
# my first program
name = input("Your name? ")
print("Hello,", name)
```

Output

```
Hello, Asha
```

input()

Reads what the user types and hands it back as text.

name = ...

A variable stores the value. No type needs to be declared.

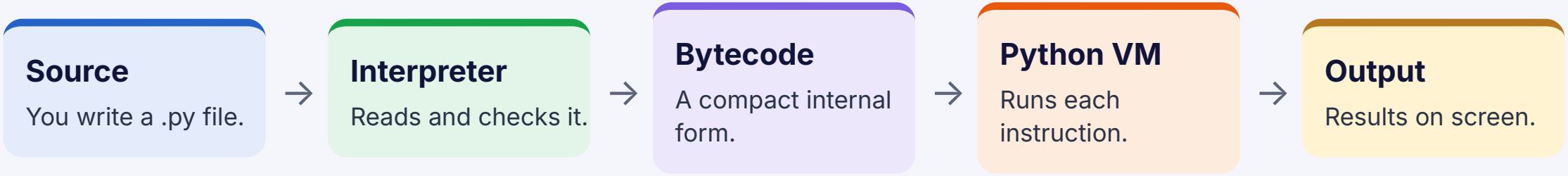
print()

Shows the result on screen, with items joined by a space.

~4 min

From Source Code to Output

What happens between pressing Run and seeing a result.



Reading it end to end

Your file is translated into bytecode, and the Python virtual machine executes it one instruction at a time. If a line has an error, Python stops there and tells you which line, which makes mistakes quick to find.

~3 min

Where Python Is Used

Six areas, one language.

Web development

Django and Flask power sites and APIs.

Data science

Pandas and NumPy clean and analyse data.

AI and machine learning

TensorFlow and PyTorch train models.

Automation

Short scripts replace repetitive tasks.

IoT and hardware

MicroPython runs on small boards.

Education

A common first language in schools.

The same basics you learn today carry over to every area above.

ONE-LINE TAKEAWAY

Python lets you say what you mean in fewer lines, which is why it is a great first language.

Readable **Interpreted** **Versatile** **Free**

Up next: Topic 2 · Variables and Data Types

PYTHON FOUNDATIONS · TOPIC 2

20 MIN

Types and Operations

Value → Type → Operator → Result

`int` `float` `str` `bool` `list`

~3 min

Every Value Has a Type

The type decides what you are allowed to do with a value.

In simple words

A type tells Python what kind of thing a value is. Numbers can be added, text can be joined, and Python picks the type from the value itself, so you never declare it.

Five types, in order

1. **int: whole numbers**
2. **float: decimals**
3. **str: text**
4. **bool: True or False**
5. **list: ordered items**

Ask Python for the type of any value with type(). Operations then work on those types.

~4 min

Core Data Types

Six built-in types cover most beginner programs.

int

Whole numbers, such as **42** or **-7**.

float

Numbers with a decimal point, such as **3.14**.

str

Text in quotes, such as **"Hello"**.

bool

Only **True** or **False**, used for decisions.

list

Ordered, changeable items, such as **[1, 2, 3]**.

None

A special value that means no value yet.

Numbers, text and booleans hold one value. A list holds many.

~4 min

Types in Action

Python works out each type, then applies the operator.

```
# three values, three types
age = 20
price = 99.5
name = "Asha"
print(type(age), age + price)
```

Output

```
<class 'int'> 119.5
```

type()

Reports the type of any value or variable.

int + float

Mixing the two gives a float: 20 + 99.5 is 119.5.

str + str

Joins text: "Hi " + "Asha" gives "Hi Asha".

~4 min

Operators at a Glance

Four families, each with a clear job.

Arithmetic

+ - * / // % **

Maths. / gives a float, // drops the decimals, % is the remainder.

Comparison

== != < > <= >=

Compare two values and give True or False.

Logical

and or not

Combine conditions: both, either, or the opposite.

Assignment

= += -= *=

Store a value, or update a variable in place.

Comparison and logical operators always give a bool: True or False.

~4 min

Evaluating an Expression

Python follows a fixed order, not left to right.

Expression
`2 + 3 * 4`

→

Multiply first
`3 * 4 = 12`

→

Then add
`2 + 12 = 14`

→

Result
`14 (int)`

Order of operations
Brackets first, then **, then * / // %, then + -, then comparisons, then not, and, or. Use brackets whenever the order is not obvious: (2 + 3) * 4 is 20.

ONE-LINE TAKEAWAY

The type decides which operations make sense, and the operator decides what happens next.

Types | Operators | Precedence | type()

Up next: Topic 3 · Control Flow

PYTHON FOUNDATIONS

25 MIN

Core Data Types and Files

Object → Operations → Result

Numbers

Strings

Lists

Tuples

Dictionaries

Files

Numeric types

~3 min

Numbers

Integers and floats cover everyday arithmetic.

```
a = 7
b = 2
print(a / b, a // b)
print(a % b, a ** b)
```

Output

```
3.5 3
1 49
```

Arithmetic as in maths

+ - * / // % ** work on any numbers. / always gives a float, and // keeps only the whole part.

Useful built-ins

abs(), round(), pow(), min() and max(), plus the math module for roots and trigonometry.

Python integers have no fixed size limit, so very large whole numbers still work.

~4 min

Strings

Text in quotes, read by position.

```
s = "Python"
print(s[0], s[-1], s[1:4])
print(s.upper(), len(s))
print("Py" + "thon", s * 2)
```

Output

```
P n yth
PYTHON 6
Python PythonPython
```

Indexing and slicing

s[0] is the first character, s[-1] the last, and s[1:4] a slice from 1 up to 3.

Methods

upper(), lower(), strip(), split() and replace() each return a new string.

Strings cannot be changed in place; every method gives back a new string.

~4 min

Lists

Ordered items that you can change.

```
nums = [3, 1, 2]
nums.append(5)
nums.sort()
print(nums, len(nums))
print(nums[0], nums[-1])
```

Output

```
[1, 2, 3, 5] 4
1 5
```

Ordered and changeable

Items keep their position, and you can add, remove or replace them after the list is made.

Common methods

append(), insert(), remove(), pop() and sort(). Indexing and slicing work as in strings.

A list can hold mixed types, even other lists.

~3 min

Tuples

Ordered items that stay fixed.

```
pt = (4, 7)
x, y = pt
print(x, y, len(pt))
pt[0] = 9 # error
```

Output

```
4 7 2
TypeError
```

Ordered but fixed

Written with round brackets. Once created, items cannot be changed, added or removed.

When to use one

Fixed groups such as coordinates, and for returning several values. Unpacking splits one into names.

A one-item tuple needs a trailing comma: (5,).

~4 min

Dictionaries

Values found by key, not by position.

```
d = {"name": "Asha", "age": 20}
d["city"] = "Pune"
print(d["name"], d.get("zip"))
print(list(d.keys()))
```

Output

```
Asha None
['name', 'age', 'city']
```

Key and value pairs

Look up a value by its key. Keys must be unique and unchangeable, such as strings or numbers.

Common methods

get(), keys(), values(), items() and pop(). Assigning to a new key adds it.

Since Python 3.7 a dictionary remembers the order in which keys were added.

~4 min

Files

Reading and writing text on disk.

```
with open("notes.txt", "w") as f:
    f.write("Hello\n")
with open("notes.txt") as f:
    print(f.read())
```

Output

```
Hello
```

Open, use, close

open(name, mode) returns a file object. Modes: r reads, w writes, a appends.

Use with

The with block closes the file automatically, even when an error happens.

Mode w erases the old contents. Use a to add to the end instead.

~3 min

Numeric Types

int, float and complex, and how to convert.

```
a = 5 # int
b = 5.0 # float
c = 2 + 3j # complex
print(int(3.9), float(2))
print(0.1 + 0.2)
```

Output

```
3 2.0
0.30000000000000004
```

Three numeric types

int for whole numbers, float for decimals, complex for a number with an imaginary part like 3j.

Conversion and precision

int() cuts off decimals and float() adds them. Floats are stored in binary, so tiny rounding errors appear.

For exact decimal arithmetic, such as money, use the decimal module.

PYTHON FOUNDATIONS

20 MIN

Dynamic Typing

Name → Reference → Object → Type

Names

Objects

Rebinding

type()

isinstance()

~3 min

Variables Are Names, Objects Have Types

No declarations: Python learns the type from the value.

In simple words

Writing x = 10 does not put a number in a box called x. It makes the name x refer to an object that is an int. Later, x can refer to text or a list instead.

Five ideas, in order

1. **Assignment creates the name**
2. **The name refers to an object**
3. **The type lives in the object**
4. **Reassign to any other object**
5. **Check with type()**

Dynamic typing means types are decided while the program runs, not before.

~4 min

Dynamic and Static Typing

What changes when you do not declare types.

No declarations

Write x = 5 instead of declaring an int first. Less code to write and easy to try ideas.

Checked at run time

A wrong operation is reported only when that line runs, as a TypeError.

Names are flexible

The same name can refer to an int now and a string later. Use clear names to avoid confusion.

Languages like C and Java check types before running. Python trades that early check for flexibility.

~4 min

Types Change With the Value

One name, three objects over time.

```
x = 10
print(type(x))
x = "ten"
print(type(x))
```

Output

```
<class 'int'>
<class 'str'>
```

Names and objects

x is a label. 10 and "ten" are objects, and each object knows its own type.

Rebinding

x = "ten" points the name at a new object. The old one is freed when nothing refers to it.

The type lives in the object, never in the variable name.

~4 min

Shared References

Two names can point at the same object.

```
a = [1, 2]
b = a
b.append(3)
print(a)
```

Output

```
[1, 2, 3]
```

Two names, one object

b = a copies the reference, not the list, so a change through b also shows through a.

Make a real copy

Use a.copy() or a[:] when you want a separate list.

Numbers, strings and tuples cannot change in place, so sharing them is safe.

~4 min

Type Checks at Run Time

Python checks each operation as it happens.

Run a line
Python reaches it.

→

Look at objects
It reads each type.

→

Allowed?
Does the type support it?

→

Result
A value or a TypeError.

Seeing it happen

"5" + 5 fails with a TypeError only when that line runs, while int("5") + 5 gives 10. Use isinstance(x, int) to test a type. Type hints can document types, but Python itself does not enforce them.

ONE-LINE TAKEAWAY

In Python the type belongs to the object, not the variable, so one name can refer to anything.

Names | Objects | Rebinding | isinstance()

PYTHON FOUNDATIONS · 20 MIN

Assignments, Expressions and Statements

Statement

A complete instruction

`x = 5`

Expression

Anything with a value

`5 + 2`

Assignment

Binds a name

`y = x + 2`

~3 min

Three Building Blocks

Every Python program is made of these ideas.

1

Statements

One complete instruction that Python carries out.

x = 5

2

Expressions

A piece of code that works out to a value.

5 + 2

3

Assignments

A statement that binds a name to a value.

y = x + 2

Statements do things. Expressions produce values. Assignments connect the two.

~4 min

Kinds of Statements

Each statement tells Python to perform one action.

Kind	Example	What it does
Assignment	x = 5	Binds a name to a value
Expression statement	print(x)	Runs an expression for its effect
Conditional	if x > 3:	Chooses which code runs
Loop	for i in range(3):	Repeats a block of code
Import	import math	Loads a module to use

A program is a list of statements, which Python runs one after another.

~4 min

Expressions

Type one into the interpreter and it answers with its value.

```
>>> 2 + 3 * 4
14
>>> len("hello")
5
>>> 7 > 3 and 2 > 1
True
>>> [n * 2 for n in range(3)]
[0, 2, 4]
```

Always a value

Every expression works out to one value, and that value has a type.

Built from parts

Literals, names, operators and function calls combine into bigger expressions.

An expression can be used wherever a value is needed, such as on the right of `=` .

~4 min

Assignments

The statement that creates and updates variables.

```
x = 5
y = x + 2
a, b = 1, 2
a, b = b, a
x += 3
print(x, y, a, b)
```

Output

```
8 7 2 1
```

Right side first

Python works out the expression on the right, then binds the name on the left.

Several forms

Plain: x = 5. Unpacking: a, b = 1, 2. Augmented: x += 3. Chained: a = b = 0.

~3 min

Expression or Statement?

A quick way to tell them apart.

Code	Kind	Has a value?
5 + 2	Expression	Yes: 7
x = 5	Assignment statement	No
print("hi")	Expression (a call)	Yes: None
x > 3	Expression	Yes: True or False
if x > 3:	Compound statement	No

An assignment never gives a value back, so it cannot sit inside another expression.

ONE-LINE TAKEAWAY

Expressions produce values, and assignment statements give those values names.

Statements | Expressions | Assignments

PYTHON FOUNDATIONS · 25 MIN

Loops, Iterations and Comprehensions

Loop

Repeat a block

```
for i in range(3):
```

Iteration

Step through items

```
next(it)
```

Comprehension

Build a collection

```
[n * n for n in nums]
```

~3 min

Three Ways to Repeat Work

Each one solves a different part of the same problem.

1

Loops

Run a block of code again and again.

for while

2

Iteration

Step through the items of a collection one at a time.

range zip

3

Comprehensions

Build a new collection from a loop in one line.

[n * 2 for n in nums]

Loops repeat actions, iteration hands over the items, and comprehensions are a short form of a loop.

~4 min

The for Loop

Visit each item of a collection in turn.

```
total = 0
for n in [4, 7, 9]:
    total += n
print(total)
for i in range(3):
    print(i, end=" ")
```

Output

```
20
0 1 2
```

for visits each item

The loop variable takes each item in turn, from a list, string, dictionary or file.

range() counts

range(3) gives 0, 1, 2. range(1, 6, 2) gives 1, 3, 5.

~4 min

The while Loop

Repeat for as long as a condition stays true.

```
n = 3
while n > 0:
    print(n)
    n -= 1
print("Go!")
```

Output

```
3
2
1
Go!
```

Repeat while true

The condition is checked before every pass. If it never becomes False, the loop never ends.

break and continue

break leaves the loop at once. continue skips to the next pass.

~4 min

Iteration Tools

Built-in helpers that feed a for loop.

Tool	Example	What you get
range	range(1, 4)	1, 2, 3
enumerate	enumerate(['a', 'b'])	(0, 'a'), (1, 'b')
zip	zip([1, 2], ['a', 'b'])	(1, 'a'), (2, 'b')
reversed	reversed([1, 2, 3])	3, 2, 1
sorted	sorted([3, 1, 2])	A new list: 1, 2, 3

An iterable is anything a for loop can step through. An iterator hands out items one at a time with next().

~4 min

Comprehensions

A loop that builds a collection in one expression.

```
nums = [1, 2, 3, 4]
sq = [n * n for n in nums]
ev = [n for n in nums if n % 2 == 0]
d = {n: n * n for n in nums}
print(sq, ev)
print(d)
```

Output

```
[1, 4, 9, 16] [2, 4]
{1: 1, 2: 4, 3: 9, 4: 16}
```

Same loop, shorter

[expression for item in items if test] builds a new list in a single line.

Other kinds

Curly brackets with key: value build a dictionary. Curly brackets with one value build a set.

ONE-LINE TAKEAWAY

Loops repeat the work, iteration supplies the items, and comprehensions say it in one line.

for · while · range() · comprehension

PYTHON FOUNDATIONS · 30 MIN

Python Functions

Basics
Define and call
`def greet(name):`

Scopes
Where names live
`global local`

Arguments
Pass values in
`f(1, key=2)`

Advanced
Functions as values
`lambda n: n + 1`

~3 min

Four Ideas About Functions

From a first def to functions that return functions.

1

Basics

Define a reusable block and call it by name.

def return

2

Scopes

Rules for where a name can be seen.

global nonlocal

3

Arguments

Pass values by position, name or default.

***args **kwargs**

4

Advanced

Treat functions as values you can pass around.

lambda closure

A function packages a job so you can write it once and reuse it everywhere.

~4 min

Function Basics

Define once, call as often as you like.

```
def area(w, h=1):
    """Return w times h."""
    return w * h
print(area(3, 4))
print(area(5))
```

Output

```
12
5
```

def and return

def names the function and its parameters. return sends a value back. Without it the result is None.

Calling it

Write the name with brackets and arguments. The same function can be called as many times as needed.

~4 min

Scopes

Where a name can be seen and changed.

```
x = "global"
def show():
    x = "local"
    print(x)
show()
print(x)
```

Output

```
local
global
```

LEGB lookup

Python looks in the Local, Enclosing, Global and then Built-in scopes, in that order.

global and nonlocal

Assigning inside a function makes a local name. Use global or nonlocal to change an outer one.

~4 min

Function Arguments

Five ways to hand values to a function.

Kind	Example	Meaning
Positional	f(1, 2)	Matched by order
Keyword	f(b=2, a=1)	Matched by name
Default	def f(a, b=0)	Used when omitted
*args	def f(*args)	Extra positional values, as a tuple
**kwargs	def f(**kw)	Extra keyword values, as a dict

A mutable default such as b=[] is shared between calls, so use None and create the list inside.

~5 min

Advanced Functions

Functions are values like any other.

```
sq = lambda n: n * n
print(list(map(sq, [1, 2, 3])))
def adder(k):
    return lambda n: n + k
add5 = adder(5)
print(add5(10))
```

Output

```
[1, 4, 9]
15
```

Functions are values

A function can be stored, passed in and returned. lambda writes a small one inline.

Closures and decorators

An inner function remembers outer variables. A decorator wraps a function to add behaviour.

ONE-LINE TAKEAWAY

A function packages a job, scope decides who sees which names, and arguments carry the data in.

def · scope · *args · lambda

PYTHON FOUNDATIONS · 35 MIN

Modules and Packages

Modules

One file, one namespace

```
import mymath
```

Packages

Folders of modules

```
import shop.cart
```

Advanced

Control how code loads

```
importlib
```

~3 min

Three Layers of Code Reuse

From a single file to a whole library.

1

Module basics

Put code in a .py file, then import it anywhere.

import mymath

2

Packages

Group modules in folders and import them by path.

import shop.cart

3

Advanced topics

Inspect, reload and load modules by name.

importlib

Every .py file is already a module, so reusing code starts with import.

~4 min

Coding a Module

Any .py file can be imported by its name.

```
# mymath.py
PI = 3.14159
def area(r):
    return PI * r * r
```

main.py

```
import mymath
print(mymath.area(2))
12.56636
```

One file, one namespace

Names defined in mymath.py are reached as mymath.name, so they cannot clash with names elsewhere.

Import runs the file

The first import executes the whole file from top to bottom and keeps the result.

~4 min

Import Forms

Four ways to bring a module into your code.

Form	Example	Effect
Plain import	import math	Use math.sqrt(4), names stay grouped
From import	from math import sqrt	Use sqrt(4) directly
Alias	import math as m	Use a shorter name: m.sqrt(4)
Import all	from math import *	Copies every public name, so avoid it

A module's code runs only on the first import. Later imports reuse the loaded module.

~4 min

How Python Finds a Module

The search runs the same way for every import.

Check the cache

sys.modules holds loaded modules.

→

Built-in modules

Part of the interpreter itself.

→

Search sys.path

Folders and zip files, in order.

→

Load or fail

Runs it, or raises ModuleNotFoundError.

Order matters

sys.path starts with the script's folder, then PYTHONPATH, then the standard library and site-packages. The first match wins, so a file named like a standard module can hide it.

~4 min

Script or Module?

One file that works both ways.

```
# tool.py
def main():
    print("running")
if __name__ == "__main__":
    main()
```

python tool.py

```
running
```

import tool

```
(nothing printed)
```

The __name__ check

__name__ is "__main__" when the file is run directly, and the module name when it is imported.

Why use it

It keeps demo or test code from running every time another file imports the module.

~4 min

Packages

A folder of modules you import by path.

```
shop/
    __init__.py
    cart.py
    pricing/
        __init__.py
        tax.py
```

main.py

```
import shop.cart
from shop.pricing import tax
```

A folder of modules

A regular package has an __init__.py file, which runs when the package is first imported.

Dotted imports

The dots follow the folders: shop.pricing.tax is the file shop/pricing/tax.py.

~4 min

Relative Imports

Refer to neighbours without the full path.

```
# shop/pricing/tax.py
from . import rates
from .. import cart
from ..cart import total
```

Dots mean position

One dot is the current package. Two dots are the parent package.

Only inside packages

Relative imports work in modules that belong to a package, not in a script run directly.

~4 min

Module Tools

Modules are objects, so you can inspect them.

Tool	What it does
dir(m)	Lists the names inside a module
m.__file__	Shows the path it was loaded from
importlib.reload(m)	Runs the module's code again
__all__	Names exported by from m import *
sys.modules	Dictionary of modules already loaded

> Reloading is handy while experimenting, but a restart is the safe way to pick up changes.

~4 min

Loading by Name

Import a module from a string.

```
import importlib
m = importlib.import_module('math')
print(m.sqrt(16))
import sys
print('math' in sys.modules)
```

Output

```
4.0
True
```

Import by name

import_module() loads a module from a string, which suits plugins and settings files.

Circular imports

If two modules import each other, one is only partly loaded. Move shared code into a third module.

ONE-LINE TAKEAWAY

A module is a file you can import, a package is a folder of modules, and sys.path tells Python where to look.

import · __init__.py · sys.path · importlib

PYTHON FOUNDATIONS · 40 MIN

Classes, Operator Overloading and Design

Classes

Objects with behaviour

`class Dog:`

Operators

Make + and [] work

`__add__`

Design

Choose the right shape

`self.engine`

~3 min

Three Parts of Object-Oriented Python

From one class to a well-shaped set of classes.

1

Classes

Bundle data and behaviour into your own types.

class self

2

Operator overloading

Let +, == and [] work on your objects.

__add__ __eq__

3

Class design

Choose inheritance, composition and clear jobs.

has-a is-a

A class is a blueprint, and each object built from it keeps its own data.

~5 min

Defining a Class

A blueprint, and objects built from it.

```
class Dog:
    def __init__(self, name):
        self.name = name
    def bark(self):
        return self.name + " says woof"
d = Dog("Rex")
print(d.bark())
```

Output

```
Rex says woof
```

Class and instance

The class is the blueprint. Each call such as Dog("Rex") builds an instance with its own data.

self

self is the instance a method was called on. Python passes it in automatically.

~4 min

Instance and Class Attributes

Data kept per object, and data shared by all.

```
class Dog:
    species = "dog"
    def __init__(self, name):
        self.name = name
a = Dog("Rex")
b = Dog("Fido")
print(a.species, b.name)
```

Output

```
dog Fido
```

Instance attributes

Set on self, so every object keeps its own copy.

Class attributes

Set in the class body and shared by every instance.

~4 min

Inheritance

A subclass reuses and overrides its parent.

```
class Animal:
    def speak(self):
        return "..."
class Cat(Animal):
    def speak(self):
        return "meow"
print(Cat().speak())
```

Output

```
meow
```

Subclass

Cat(Animal) inherits everything from Animal and can override any method.

super()

super().method() calls the parent version, which is handy when extending __init__.

~4 min

Operator Overloading

Special methods connect Python syntax to your class.

Expression	Method called	Used for
a + b	__add__	Addition
a == b	__eq__	Equality
a < b	__lt__	Ordering
len(a)	__len__	Size
str(a)	__str__	Readable text

These double-underscore methods are called by Python, not by you directly.

~4 min

Overloading +

Teach your class how to add.

```
class Money:
    def __init__(self, n):
        self.n = n
    def __add__(self, o):
        return Money(self.n + o.n)
    def __repr__(self):
        return f"Money({self.n})"
print(Money(5) + Money(3))
```

Output

```
Money(8)
```

__add__ runs for +

Money(5) + Money(3) becomes Money(5).__add__(Money(3)).

__repr__ and __str__

repr gives developer text and is the fallback for print. str gives friendly text when defined.

~4 min

Overloading []

Make an object behave like a sequence.

```
class Team:
    def __init__(self, *p):
        self.p = list(p)
    def __getitem__(self, i):
        return self.p[i]
t = Team("Ana", "Raj")
print(t[1])
for m in t: print(m, end=" ")
```

Output

```
Raj
Ana Raj
```

Indexing with []

__getitem__ makes t[1] work.

Free iteration

Python tries 0, 1, 2 and stops at IndexError, so a for loop works too.

~4 min

Choosing a Design

Two questions settle most class relationships.

Ask	Choose	Example
Is it a kind of X?	Inheritance	class Cat(Animal)
Does it have an X?	Composition	self.engine = Engine()
Same action, different class?	Polymorphism	pet.speak()

A common guideline is to prefer composition unless a true is-a relationship exists.

~4 min

Composition

A class that holds other objects.

```
class Engine:
    sound = "vroom"
class Car:
    def __init__(self):
        self.engine = Engine()
print(Car().engine.sound)
```

Output

```
vroom
```

Has-a relationship

A Car has an Engine. Change the engine class and Car keeps working.

Focused classes

Give each class one clear job, a small public interface and details kept inside.

~4 min

Dataclasses

Less boilerplate for classes that hold data.

```
from dataclasses import dataclass
@dataclass
class Point:
    x: int
    y: int
print(Point(1, 2))
print(Point(1, 2) == Point(1, 2))
```

Output

```
Point(x=1, y=2)
True
```

Written for you

@dataclass generates __init__, __repr__ and __eq__ from the listed fields.

Good for data holders

Use it for classes that mainly store values, such as points or records.

ONE-LINE TAKEAWAY

Classes bundle data with behaviour, special methods make them feel built in, and good design keeps each one focused.

class self __add__ composition

PYTHON FOUNDATIONS · 40 MIN

Python Exceptions

Basics
Catch and recover
`try / except`

Details
else, finally, raise
`finally`

Objects
Errors are classes
`except E as e`

Design
Errors as an API
`class AppError`

~3 min

Four Parts of Error Handling

From catching one error to designing for failure.

1

Basics

Catch an error and keep the program running.

try except

2

Coding details

Add else, finally and your own raise.

else finally

3

Exception objects

Errors are instances of classes in a hierarchy.

as e args

4

Designing

Plan which errors your code raises and handles.

AppError

> An exception is Python's way of saying something went wrong, and it can be handled.

~4 min

What an Exception Is

An event that interrupts normal flow.

```
nums = [1, 2, 3]
print(nums[5])
```

Output

```
Traceback (most recent call last):
IndexError: list index out of range
```

An event, not a value

An exception interrupts normal flow. If nothing handles it, the program stops and prints a traceback.

Common ones

ValueError, TypeError, KeyError, IndexError and ZeroDivisionError appear often.

~4 min

try and except

Run risky code, and recover if it fails.

```
try:
    n = int("abc")
except ValueError:
    n = 0
print(n)
```

Output

```
0
```

try and except

Code in try runs first. If it raises a matching exception, the except block runs instead.

Match specific errors

Name the exceptions you expect. A bare except hides real bugs.

~4 min

Common Exceptions

What raises them, with one example each.

Exception	Raised when	Example
ValueError	Right type, wrong value	int("abc")
TypeError	Wrong type for an operation	"5" + 5
KeyError	A dictionary key is missing	d["x"]
IndexError	A list index is out of range	[1][3]
ZeroDivisionError	Dividing by zero	1 / 0

Read the last line of a traceback first, because it names the error and the reason.

~4 min

else and finally

Code for success, and code that always runs.

```
try:
    n = int("42")
except ValueError: print("bad")
else: print("ok", n)
finally: print("done")
```

Output

```
ok 42
done
```

else and finally

else runs only when no exception happened. finally runs every time, even after a return.

Order of events

try runs first, then except if it failed or else if it did not, and finally last.

~4 min

Raising Exceptions

Signal a problem on purpose.

```
def set_age(a):
    if a < 0:
        raise ValueError("age below 0")
try:
    set_age(-1)
except ValueError as e:
    print("caught:", e)
```

Output

```
caught: age below 0
```

raise

raise ValueError(msg) reports a problem. A bare raise inside except re-raises the same error.

as e

Binds the exception object to a name so you can print or inspect it.

~4 min

Handling Several Errors

One try, more than one handler.

```
try:
    x = {"a": 1}["b"]
except (KeyError, IndexError):
    print("missing")
except Exception:
    print("something else")
```

Output

```
missing
```

Several handlers

Python checks except clauses from top to bottom and runs the first one that matches.

Order matters

Put specific exceptions before general ones such as Exception, or the general one catches everything.

~4 min

Cleanup With with

Release resources even when errors happen.

```
with open("data.txt") as f:
    data = f.read()
print(f.closed)
```

Output

```
True
```

with closes it

The with block releases the file even if an exception is raised inside it.

A shorter try/finally

with replaces a try and finally pair whenever an object supports cleanup.

~4 min

The Exception Hierarchy

Exceptions are classes, so they have parents.

Class	Includes
BaseException	Everything, including KeyboardInterrupt and SystemExit
Exception	The normal errors your programs handle
ArithmeticError	ZeroDivisionError and OverflowError
LookupError	KeyError and IndexError
OSError	FileNotFoundError and PermissionError

Catching a parent class also catches every one of its children.

~4 min

Exception Objects

What an exception carries with it.

```
try:
    int("abc")
except ValueError as e:
    print(type(e).__name__)
    print(e)
```

Output

```
ValueError
invalid literal for int() with base 10:
'abc'
```

Instances of classes

Every exception is an object, so it carries data such as its message in e.args.

Chaining

raise NewError from e links a new error to the original cause in the traceback.

~4 min

Custom Exceptions

Give your own errors clear names.

```
class AppError(Exception): pass
class NotFound(AppError): pass
try:
    raise NotFound("id 7")
except AppError as e:
    print(type(e).__name__, e)
```

Output

```
NotFound id 7
```

Subclass Exception

A custom class gives an error a clear name, and can carry extra data if needed.

A shared base class

One base such as AppError lets callers catch every error from your code at once.

~4 min

Designing With Exceptions

Habits that keep error handling honest.

Guideline	Why
Catch specific errors	A bare except hides bugs
Do not swallow errors	Log or re-raise instead of pass
Raise early	Fail near the cause, with a clear message
Use with or finally	Always release files and connections
Try first, then handle	Python style: ask forgiveness, not permission

> Handle an error only where you can do something useful about it.

ONE-LINE TAKEAWAY

Exceptions are objects you can catch, raise and design, so a failure becomes something your code can handle.

try except finally raise

PYTHON FOUNDATIONS · 45 MIN

Parallel System Tools

Threads
Share memory
`threading`

Processes
Use many cores
`multiprocessing`

Programs
Run other tools
`subprocess`

Messages
Pass data safely
`Queue`

~3 min

Four Tools for Doing Several Things at Once

Pick the tool that matches the kind of waiting.

1

Threads

Light tasks that share memory and wait on I/O.

threading

2

Processes

Separate memory and real use of several cores.

multiprocessing

3

Other programs

Start, feed and read other command-line tools.

subprocess

4

Exit codes

Report success or failure to whoever started you.

sys.exit

> Parallel work means several tasks make progress at the same time.

~4 min

Choosing a Parallel Tool

Match the tool to what your program waits for.

Goal	Tool	Typical use
Wait on network or disk	threading	Downloads, file reads
Heavy calculation	multiprocessing	Number crunching on many cores
Run another program	subprocess	Call a command-line tool

Threads share memory. Processes do not, so they need queues or pipes to talk.

~4 min

Starting a Thread

Run a function alongside the main program.

```
import threading
def work(n):
    print("task", n)
t = threading.Thread(target=work, args=(1,))
t.start()
t.join()
```

Output

```
task 1
```

start and join

start() launches the thread. join() waits for it to finish.

Shared memory

Threads share variables, so protect shared data with a lock.

~4 min

Locks and the GIL

Keep shared data safe, and know the limits.

```
import threading
count = 0
lock = threading.Lock()
def add():
    global count
    with lock:
        count += 1
```

Lock

A lock lets one thread at a time change shared data, so updates are not mixed up.

The GIL

In the standard CPython build one thread runs Python code at a time. Threads help with waiting, not heavy calculation.

~4 min

Using Several Processes

A pool of workers on separate cores.

```
from multiprocessing import Pool
def square(n):
    return n * n
if __name__ == "__main__":
    with Pool(2) as p:
        print(p.map(square, [1, 2, 3]))
```

Output

```
[1, 4, 9]
```

Separate processes

Each process has its own memory and interpreter, so heavy work can use several cores.

The main guard

On Windows and macOS children re-import your file, so keep start-up code under the __main__ check.

~4 min

Passing Messages

Processes exchange data through a queue.

```
from multiprocessing import Process, Queue
def work(q):
    q.put("hi")
if __name__ == "__main__":
    q = Queue()
    p = Process(target=work, args=(q,))
    p.start()
    print(q.get())
```

Output

```
hi
```

Queue

A queue is a safe channel between processes. One side puts items in and the other takes them out.

Why pass messages

Processes do not share variables, so they swap data through queues, pipes or files.

~4 min

Threads or Processes?

The main differences side by side.

Feature	Threads	Processes
Memory	Shared	Separate
Start-up cost	Low	Higher
Heavy calculation	Limited by the GIL	Uses several cores
Sharing data	Directly, with locks	Queues and pipes
A crash	Can stop everything	Stays inside one process

> Use threads for waiting and processes for calculating.

~4 min

Running Another Program

Start a command and read its result.

```
import subprocess
r = subprocess.run(
    ["python", "-c", "print(1 + 2)"],
    capture_output=True, text=True)
print(r.returncode, r.stdout.strip())
```

Output

```
0 3
```

run()

subprocess.run starts a program, waits for it and returns the result.

Pass a list

Give the command as a list, not one string, so spaces and quotes cause no trouble and shell injection is avoided.

~4 min

Ways to Start Programs

Older tools you will still meet in code.

Tool	Example	Note
os.system	os.system('dir')	Runs in a shell and returns the exit status
os.popen	os.popen('dir')	Older way to read a command's output
subprocess.run	subprocess.run([...])	The recommended choice today
os.fork	os.fork()	Unix only, copies the current process

Prefer subprocess for new code, because it handles arguments, output and errors safely.

~4 min

Forking a Process

Unix only: copy the running program.

```
import os
pid = os.fork()
if pid == 0:
    print("child")
    os._exit(0)
os.wait()
print("parent")
```

Output

```
child
parent
```

A copy of the process

fork() returns twice: 0 in the child, and the child's id in the parent.

Unix only

Windows has no fork. Use multiprocessing for code that must run on both.

~4 min

Exit Codes and Signals

How a program tells others how it ended.

Item	Meaning
0	Success
Any other number	Failure, and the number says why
sys.exit(n)	Leave with exit code n
os._exit(n)	Leave at once, without cleanup
signal.signal	Handle signals such as SIGINT and SIGTERM

A parent reads the child's exit code to learn whether the job worked.

ONE-LINE TAKEAWAY

Use threads for waiting, processes for calculating, and subprocess to run other programs.

threading multiprocessing subprocess Queue